FlexForge — Build Your Application

You build an app by writing a config bundle (YAML), not code. Worked example: a real NBFC lending app.

The mental model

```
your app  =  one YAML config bundle  -->  FlexForge engine  -->  a running backend
            (entities, fields, auth,                          (APIs + UI + DB, no code)
             flows, endpoints)
```

No per-app source code. Change the config and restart. Run a different app by pointing the same engine at a different bundle:

```
FLEXFORGE_CONFIG_PATH=examples/lending-app.yaml mvn spring-boot:run
# open http://localhost:8080/  (Console renders YOUR app)
```

Step 1 — Data model (entities + fields)

```
appId: nbfc-lending
entities:
  - name: Borrower
    fields:
      - { name: id,       type: LONG,   pk: true }
      - { name: fullName, type: STRING, length: 200, nullable: false }
      - { name: email,    type: STRING, email: true }
```

Field types: STRING, TEXT, INT, LONG, DOUBLE, DECIMAL, BOOLEAN, DATE, TIMESTAMP, JSON, REFERENCE (relation), FILE (upload).

Step 2 — Relations (connect entities)

```
  - name: LoanApplication
    fields:
      - { name: id,         type: LONG,      pk: true }
      - { name: borrowerId, type: REFERENCE, references: Borrower, nullable: false }
      - { name: amount,     type: DECIMAL,   min: 1000, nullable: false }
```

Step 3 — Validation (no code)

```
- { name: fullName, type: STRING,  nullable: false, minLength: 2 }   # required + min length
- { name: email,    type: STRING,  email: true }                     # must be email
- { name: phone,    type: STRING,  pattern: "^[0-9]{10,15}$" }        # regex
- { name: amount,   type: DECIMAL, min: 1000, max: 5000000 }         # numeric range
```

Invalid writes return HTTP 400 with a clear errors[] list.

Step 4 — Encryption + files

```
- { name: pan,    type: STRING, encrypted: true }   # AES-256-GCM at rest, plaintext on read
- { name: kycDoc, type: FILE }                       # /api/Borrower/{id}/file/kycDoc
```

Step 5 — Auth + role rules (config-driven)

```
security:
  enabled: true
  users:
    - { username: officer, password: officer123, roles: [OFFICER] }
    - { username: manager, password: manager123, roles: [MANAGER] }
  rules:
    LoanApplication: { read: [OFFICER, MANAGER], write: [OFFICER] }
    Disbursement:    { write: [MANAGER] }
```

Same rules apply across REST, GraphQL and flows. Login: POST /auth/login, or use X-API-Key.

Step 6 — Business logic as no-code flows

Real loan auto-decisioning (create -> branch on amount -> approve/review):

```
flows:
  - name: applyLoan
    steps:
      - id: create
        type: db
        params: { op: create, entity: LoanApplication,
                  data: { borrowerId: "${input.borrowerId}", amount: "${input.amount}",
                          status: "SUBMITTED" } }
        next: decide
      - id: decide
        type: condition
        params: { left: "${input.amount}", op: "<=", right: 200000 }
        then: approve
        else: review
      - id: approve
        type: db
        params: { op: update, entity: LoanApplication, id: "${steps.create.id}",
                  data: { status: "APPROVED", score: 750 } }
        next: done
      - id: done
        type: respond
        params: { body: { applicationId: "${steps.create.id}", status: "APPROVED" } }
      - id: review
        type: respond
        params: { body: { status: "MANUAL_REVIEW" } }
```

Run: POST /flows/applyLoan/run {"borrowerId":1,"amount":150000} -> {"applicationId":1,"status":"APPROVED"} (300000 -> MANUAL_REVIEW).

Step types: db, http, condition, set, respond, log, notify, ai, math, string, list, datetime, crypto, json (60+ ops). AI steps plug in the same way.

Step 7 — Run & use it (what you get free)

- Admin Console at / (dashboard, data browser, generated forms, flow runner)
- REST CRUD + search + pagination for every entity; OpenAPI at /__meta/openapi.json
- GraphQL at /graphql (+ SDL); Realtime at /realtime/stream and /ws/events
- Audit log at /__audit; flows at /flows/{name}/run

What you did NOT write

No controllers, repositories, DTOs, SQL, migrations, auth filters, validation code, serializers, API spec, or UI. All derived from the bundle by the engine.

Build YOUR app

Copy examples/lending-app.yaml, change appId, replace entities/flows with your domain, run. The same pattern builds a clinic, CRM, marketplace, LMS — see INDUSTRY_CATALOG.md for the entities/features each vertical needs.